

VGGNet

: very deep convolutional networks for large-scale image recognition
Karen Simonyan* & Andrew Zisserman+

Abstract

- large scale image recognition setting에서 convolutional network 깊이의 효과를 조사함
- 주된 기여
 1. 매우 작은 **3x3 convolution filter**구조를 사용하는 **깊이 증가**한 네트워크로 상당한 성능 향상을 이룸 (16-19 weight layer)
 2. ImageNet Challenge 2014의 기반이 됨 (localisation track : 1위, classification track : 2위)
 3. 다른 데이터셋에서도 잘 일반화됨
- 2개의 최고 성능 ConvNet 모델 공개
 - > 컴퓨터 비전에서 깊은 visual representation 활용에대한 후속 연구 촉진

Introduction

- ConvNet

큰 규모의 이미지, 비디오 **recognition**에서 큰 성과를 얻음

<- large public 이미지 저장소(ImageNet) + GPU + 큰 규모의 분산 클러스터

- ILSVRC

고차원 얇은 특징 인코딩 ~ 깊은 ConveNet

큰 규모의 이미지 분류 시스템에 대한 **test bed** 역할을 함

Introduction

- Krizhevsky 등(2012)이 제안한 원래의 구조를 향상시키기 위한 다양한 시도

1. 첫번째 **convolutional layer**에 작은 수용가능 윈도우사이즈 + 작은 **stride** 사용

(ILSVRC-2013에서의 최고 성능 **submission**)

2. 여러 규모와 전체이미지에서 밀집하게 네트워크를 학습/테스트

- In this paper

파라미터를 고치고 **convolution layers**를 추가함-> 네트워크를 꾸준히 증가시킴

-> 모든 **layers**에서 매우 작은 **3x3 convolution filter**를 사용했기에 가능했음

Introduction

- depth의 증가

=> 훨씬 더 정확한 ConvNet구조를 얻음

- ILSVRC 분류에서 최상의 결과
- 다른 이미지 인지 데이터셋에서도 적용가능
- 상대적으로 간단한 파이프라인의 부분으로서 사용시에도 훌륭한 성능을 얻음

(파인튜닝 없이 선형 SVM으로 분류된 deep feature)

ConvNet Configurations - 1. Architecture

- 훈련시 ConvNets에 대한 input : 224x224 RGB 이미지
- 전처리 : 훈련 세트에서 계산된 평균 RGB 값을 각 픽셀에서 빼는 것만 적용
- Conv layer

3x3 conv filter 사용 (->좌/우/상/하등의 공간 개념을 포착할 수 있는 최소 크기임!)

1x1 conv filter도 사용 (-> input 채널의 linear transformation -> 차원줄이고
비선형성 증가 목적)

stride = 1, padding =1 (convolution 후에도 공간 해상도가 유지되도록!)

- pooling layer

5개의 max pooling layer에 의해 수행, conv layer 뒤에 위치

2x2 픽셀 윈도우, stride=2

ConvNet Configurations - 1. Architecture

- FC layer

convolutional layers stack 뒤에 3개의 fc layer가 붙음

첫번째 2개의 layer : 4096개의 채널

세번째 layer : 1000개의 채널 (-> 1000가지 ILSVRC분류 수행)

- soft-max layer

마지막에 적용

ConvNet Configurations - 1. Architecture

- Convolutional layers의 stack -> 각기 다른 구조, 다른 깊이를 가짐
- Fc layer의 configuration -> 모든 네트워크에서 같음
- 모든 hidden layers -> ReLU사용
- LRN 사용 X (성능 향상X, 오히려 메모리 소비와 계산 시간을 늘림)

ConvNet Configurations - 2. Configurations

- 구조는 동일, 깊이에서만 차이가 있음
- network A~E (-> 깊이 증가)

network A : 11 weight layer

network E : 19 weight layer

- conv layer의 width -> 작게 시작

(64->max pooling 거칠 때마다 2배씩
증가 -> 512)

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

ConvNet Configurations - 2. Configurations

- 각 configuration의 파라미터 수

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

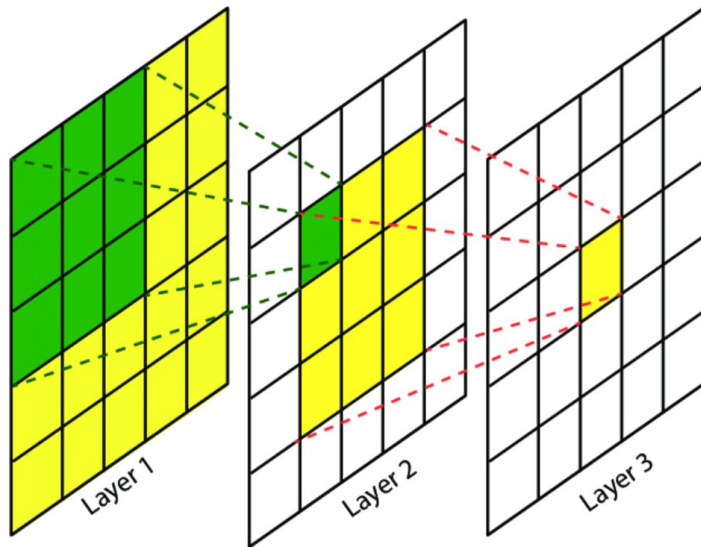
- large depth에도 네트워크에서 weight의 수는 더 큰 conv filter를 사용하는 얇은 네트워크보다 크지 않음
- (144M weights in (Sermanet et al., 2014))

ConvNet Configurations - 3. Discussion

- 우리의 conv net configuration -> first conv에서 작은 3x3 receptive field를 사용
- 2개의 3x3 conv층을 pooling 없이 연속 적용 -> 5x5 수용영역 가짐

3개의 such layers

→ 7x7 effective receptive field를 가짐



ConvNet Configurations - 3. Discussion

Q. 왜 single 7x7이 아니라 stack of 3x3 conv.layer를 사용하는가?

1. 3개의 non-linear rectification later 도입 가능 -> 더 정밀한 decision function 가능
2. 파라미터 수 줄일 수 있음 (7x7 conv.filter에 regularisation 부과와 같은 효과)

3개의 3x3 conv stack의 input,output 둘다 C 채널 가정

3x3 stack : $3 \times 3 (\text{필터수}) \times C (\text{input 채널 수}) \times C (\text{output 채널 수} = \text{filter 수}) \times 3 (\text{연속 3번 적용})$

-> $27C^2$

single 7x7 : $7 \times 7 \times C \times C \rightarrow 49C^2$

ConvNet Configurations - 3. Discussion

- 1x1 conv.layer의 도입

receptive field에 영향을 미치지 않고 결정함수의 비선형성만 증가 가능

($\leftarrow$ ReLU)

→ 최근 Lin 외 연구진의 'Network in Network' 아키텍처에서 활용된 바 있음

ConvNet Configurations - 3. Discussion

- Ciresan : 작은 conv.filter 사용, 그러나 우리에게 비해 네트워크 깊지 X
- Goodfellow : 깊은 네트워크 도로 숫자 인지 task에 적용
- GoogleNet(Szegedy) : 우리와 독립적으로 발전 되었으나 매우 깊은 ConvNet+ 작은 conv.filter기반인 것이 유사함, 네트워크 구조방식이 더 복잡, feature map의 공간적 해상도를 첫번째 layer에서 공격적으로 줄여 계산을 줄임
 - > 우리 모델이 단일 네트워크 분류정확도에서 능가함

Classification Framework - 1. Training

하이퍼파라미터 설정 (AlexNet에서 사용한 설정 Krizhevsky를 따름)

- cost function : Multinomial logistic regression objective
- optimizer : 미니 배치 경사 하강법 + momentum

batch size : 256 size

momentum = 0.9

- regularization :

weight decay -> L2 penalty multiplier = 5×10^{-4}

dropout -> 첫번째 2개의 fc layer에 적용, drop ratio = 0.5

- learning rate = 10^{-2} (val set 정확도가 더 나아지지 않으면 1/10씩 줄임)

Classification Framework - 1. Training

- 370K iterations 후에 중단 (74에폭)

-> 더 많은 파라미터와 깊은 depth에도 Krizhensky보다 더 적은 에폭수가 요구됨

Why?

- 깊은 깊이와 작은 conv filter size → 암묵적인 regularisation
- 일부 층 pre-initialization

얇은 모델(A)을 먼저 학습한 후, 그 일부 층을 깊은 모델에 초기값으로 사용

→ 학습 안정성 향상

Classification Framework - 1. Training

초기화

- pre-initialization

랜덤 초기화로도 학습가능한 configuration A로 학습

더 깊은 구조 훈련시 첫번째 4개의 conv.layer과 마지막 3개의 fc layers -> net A의 layers로 초기화

- 나머지 랜덤 초기화

weight -> 정규분포 (평균=0, 분산 10^{-2})에서 샘플링

bias -> 0으로 초기화

- 미리 초기화된 layers에 대한 learning rate 줄이지 않고 계속 학습되게 함
- 논문 제출후 weight pre-training 없이 초기화 가능함 알아냄 (<- Xavier 초기화)

Classification Framework - 1. Training

training image size

- 고정된 224x224 input image를 얻기 위해 임의로 **rescale**된 훈련이미지를 **crop**

S : 비율을 유지하여 **rescale**된 훈련이미지의 가장 작은 **side**

crop size : 224x224로 고정

- S -> 224보다 작을 수 X

S=224 : crop이 가장 짧은 변을 완전히 덮음 -> 전체 이미지 포착 가능

S>224 : crop이 이미지의 일부분만 포함 -> 작은 객체나 부분적 특징 학습 가능

Classification Framework - 1. Training

training image size

- S값 설정 방식

1. Single scale training

: 모든 학습 이미지에 대해 S를 고정된 값(256, 384)으로 설정

- S=256 이용해 먼저 네트워크를 훈련

- S=384 네트워크 훈련을 빠르게 하기위해 S=256으로 초기화된 w사용
(더 작은 learning rate 10^{-3} 사용)

Classification Framework - 1. Training

training image size

- S값 설정 방식

2. multi-scale training

: 각 training 이미지는 각각 rescale됨 특정 범위 [Smin, Smax]에서 임의로 샘플링된 S에 의해 각각 rescale됨 (Smin = 356, Smax = 512)

-> 단일 모델이 다양한 scale의 객체 인지 가능

-> scale jittering을 통한 학습 데이터 증강 효과

- 속도를 빠르게 하기 위해 S=384로 미리 훈련된 single-scale 모델을 파인튜닝함

Classification Framework - 2. Testing

- test 시에도 rescale함

Q : test scale (S와 같을 필요X, 각 S값마다 다른 Q를 적용하는게 성능 좋았음)

- Validation 수행 시의 구조

첫번째 FC layer -> 7x7 conv.layer

남은 2개의 FC layer -> 1x1 conv.layer

=> fully-convolutional net이 전체 uncropped 이미지에 적용됨

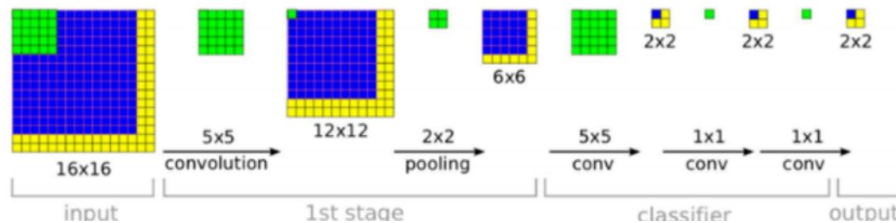
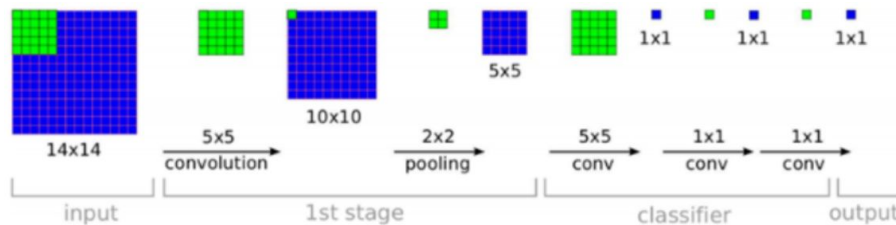
(fc layer가 1x1 conv.layer로 바뀌었기에 test시엔 uncropped image사용해도 됨!)

Classification Framework - 2. Testing

- class score map

채널 수 = class 수

공간해상도 <- 입력 이미지 크기에 따라 달라짐



Classification Framework - 2. Testing

- class score map 평균 처리 (sum-pooled)
 - > 이미지에 대한 **class score**의 고정된 크기 벡터
- **test set**도 이미지 좌우반전을 통해 증강
 - 원본 이미지와 좌우반전된 이미지의 **soft-max class** 사후확률을 평균 처리
- **multiple crops**을 **test**시에 샘플링할 필요X

Classification Framework - 2. Testing

- fully-convolutional evaluation (Dense Evaluation)

padding이 주변이미지로부터 자연스럽게 나옴

→ receptive field가 더 넓어지고 더 많은 문맥 정보를 포착 가능

- multi-crop evaluation

padding이 zero로 채워짐

=> 서로 다른 convolution boundary 처리 방식 -> dense evaluation에 상호보완적

같이 적용 시 성능이 더 향상됨 (scale당 50 crop사용 <- Szegedy와 유사하도록)

Classification Framework - 3. Implementation Details

- C++ CAffe toolbox 이용
- training/test -> 단일 시스템에서 설치된 multiple GPU에서 수행
- multiple scale로 훈련/평가
- Multi GPU 훈련 데이터 병렬 수행 활용 -> 배치 gradient 계산 후 평균 냄

GPU간 gradient 계산은 동기적 -> 단일 GPU에서 학습한 결과와 동일

- off-the-shelf 4-GPU system

데이터 병렬 처리만 사용 -> 3.75배의 속도 증가 (<-> 단일 GPU)

단일 네트워크 훈련 2~3주 걸림

4. Classification Experiments

- Dataset

ILSVRC-2012 dataset : 1000개의 class 포함

3개의 set로 나뉨짐 -> training(1.3M), validation(50K), testing(100K)

-> 대다수의 실험에서 val set을 test set으로 사용

- 분류 성능 측정 : top-1, top-5 error

4.1 Single Scale Evaluation

Single Scale Evaluation : test시 image size(Q)가 고정

<고정된 Q 설정방식>

- 고정된 S

$$Q = S \text{ ex) } 256 \rightarrow 256$$

- 스케일이 jittered된 경우 ($S \in [S_{\min}, S_{\max}]$) ex) 256, 512

$$Q = 0.5 (S_{\min} + S_{\max}) \text{ ex) } 0.5(256+512) = 384$$

4.1 Single Scale Evaluation

- 결과

Table 3: **ConvNet performance at a single test scale.**

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	25.5	8.0

4.1.Single Scale Evaluation

<네트워크 B vs 5개의 5x5 conv.layer의 얇은 네트워크>

네트워크 B의 $(3 \times 3 \text{ conv} + 3 \times 3 \text{ conv}) \times 5 \rightarrow (5 \times 5 \text{ conv}) \times 5$

=> 얇은 네트워크의 top-1 error 7%높았음 (<-> B)

=> 작은 필터와 깊은 네트워크 >> 큰 필터와 얇은 네트워크

4.2 Multi-Scale Evaluation

훈련시 S scale jittering한 게 성능이 좋았음 => test에도 적용!!

1. 각각 다른 3개의 Q값 -> rescaled test image 전체를 모델에 돌림
2. resulting class posteriors를 평균화함
 - train과 test scale 차이가 크면 -> 성능 저하
 - 고정된 S
 - > ex) $Q = \{S-32, S, S+32\}$
 - train시 scale jittering 적용 $S \in [S_{min}; S_{max}]$
 - > $\{S_{min}, 0.5(S_{min}+S_{max}), S_{max}\}$

4.2 Multi-Scale Evaluation

결과

Table 4: **ConvNet performance at multiple test scales.**

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
B	256	224,256,288	28.2	9.6
C	256	224,256,288	27.7	9.2
	384	352,384,416	27.8	9.2
	[256; 512]	256,384,512	26.3	8.2
D	256	224,256,288	26.6	8.6
	384	352,384,416	26.5	8.6
	[256; 512]	256,384,512	24.8	7.5
E	256	224,256,288	26.9	8.7
	384	352,384,416	26.7	8.6
	[256; 512]	256,384,512	24.8	7.5

4.3 Multi-Crop Evaluation

- Dense ConvNet Evaluation vs Multi ConvNet Evaluation

Table 5: **ConvNet evaluation techniques comparison.** In all experiments the training scale S was sampled from $[256; 512]$, and three test scales Q were considered: $\{256, 384, 512\}$.

ConvNet config. (Table 1)	Evaluation method	top-1 val. error (%)	top-5 val. error (%)
D	dense	24.8	7.5
	multi-crop	24.6	7.5
	multi-crop & dense	24.4	7.2
E	dense	24.8	7.5
	multi-crop	24.6	7.4
	multi-crop & dense	24.4	7.1

4.4 ConvNet Fusion

- 여러 모델의 output 합하기 -> 그들의 soft-max class posteriors 평균화
- 결과 (top ILSVRC2012)

(오직 single-scale 네트워크를 훈련)

Table 6: **Multiple ConvNet fusion results.**

Combined ConvNet models	Error		
	top-1 val	top-5 val	top-5 test
ILSVRC submission			
(D/256/224,256,288), (D/384/352,384,416), (D/[256;512]/256,384,512) (C/256/224,256,288), (C/384/352,384,416) (E/256/224,256,288), (E/384/352,384,416)	24.7	7.5	7.3
post-submission			
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), dense eval.	24.0	7.1	7.0
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), multi-crop	23.9	7.2	-
(D/[256;512]/256,384,512), (E/[256;512]/256,384,512), multi-crop & dense eval.	23.7	6.8	6.8

4.5 Comparison with the state of the art

- 기존 최고 성능과의 비교 (ILSVRC-2014challenge의 분류 task)

Table 7: **Comparison with the state of the art in ILSVRC classification.** Our method is denoted as “VGG”. Only the results obtained without outside training data are reported.

Method	top-1 val. error (%)	top-5 val. error (%)	top-5 test error (%)
VGG (2 nets, multi-crop & dense eval.)	23.7	6.8	6.8
VGG (1 net, multi-crop & dense eval.)	24.4	7.1	7.0
VGG (ILSVRC submission, 7 nets, dense eval.)	24.7	7.5	7.3
GoogLeNet (Szegedy et al., 2014) (1 net)	-	7.9	
GoogLeNet (Szegedy et al., 2014) (7 nets)	-	6.7	
MSRA (He et al., 2014) (11 nets)	-	-	8.1
MSRA (He et al., 2014) (1 net)	27.9	9.1	9.1
Clarifai (Russakovsky et al., 2014) (multiple nets)	-	-	11.7
Clarifai (Russakovsky et al., 2014) (1 net)	-	-	12.5
Zeiler & Fergus (Zeiler & Fergus, 2013) (6 nets)	36.0	14.7	14.8
Zeiler & Fergus (Zeiler & Fergus, 2013) (1 net)	37.5	16.0	16.1
OverFeat (Sermanet et al., 2014) (7 nets)	34.0	13.2	13.6
OverFeat (Sermanet et al., 2014) (1 net)	35.7	14.2	-
Krizhevsky et al. (Krizhevsky et al., 2012) (5 nets)	38.1	16.4	16.4
Krizhevsky et al. (Krizhevsky et al., 2012) (1 net)	40.7	18.2	-

Conclusion

- 매우 깊은 convolutional network를 평가함
=> 깊이가 모델의 분류 정확도에 매우 중요했음!!
- 넓은 범위의 task와 데이터셋에도 적용가능하게 일반화됨

A. Localisation

A.1 Localisation ConvNet

- 매우 깊은 ConvNet이 용-> object localisation
- 마지막 fc layer -> class score 대신 bounding box를 예측

bounding box : 4-D vector로 표현됨 (중심좌표, 넓이, 높이)

- SCR : 모든 클래스에 대해 예측을 공유 -> 마지막 layer가 4-D
- PCR : 클래스 별 따로 예측 -> 마지막 layer가 4000-D
- 마지막 bounding box 예측 layer제외하고는 ConvNet 구조 D 사용

A.1 Localisation ConvNet - training

- classification ConvNet과의 차이점
 - : logistic regression 목적함수 $\rightarrow$ Euclidean loss 로 변경
- 두개의 localisation 모델을 훈련

단일 규모 : $S=256$, $S=384$

- 초기화

learning rate : 10^{-3}

마지막 **fc layer** $\rightarrow$ 임의로 초기화되어 처음부터 훈련됨

나머지는 모두 파인튜닝함

A.1 Localisation ConvNet - testing

2가지 testing protocol

1. 네트워크 modifications을 val set에서 비교 목적

오직 정답 class에 대해서만 bounding box 예측 고려

bounding box → 네트워크를 오직 이미지 중앙 크롭에 적용 (전체 이미지X)

2. fully-fledged testing

전체 이미지에 대하여 localistion ConvNet을 조밀하게 적용

-> Sermanet의 greedy merging 과정 사용

1) 공간적으로 가까운 예측을 합함 -> 좌표 평균으로 합치기

2) 병합된 box classification ConvNet으로 얻어진 class score에 기반하여 평가 -> 가장 신뢰도 높은 box

3) 다른 scale에서 학습된 모델 예측 합집합하여 병합

A.2 Localisation Experiments

- settings comparison

1st protocol -> the best-performing localisation setting 결정

Fine-tuned layers	regression type	GT class localisation error
1st and 2nd FC	SCR	36.4
	PCR	34.3
all	PCR	33.1

A.2 Localisation Experiments

- fully-fledged evaluation

Table 9: **Localisation error**

smallest image side		top-5 localisation error (%)	
train (S)	test (Q)	val.	test.
256	256	29.5	-
384	384	28.2	26.7
384	352,384	27.5	-
fusion: 256/256 and 384/352,384		26.9	25.3

A.2 Localisation Experiments

- Comparison with state of art (in ILSVRC localisation)

Method	top-5 val. error (%)	top-5 test error (%)
VGG	26.9	25.3
GoogLeNet (Szegedy et al., 2014)	-	26.7
OverFeat (Sermanet et al., 2014)	30.0	29.9
Krizhevsky et al. (Krizhevsky et al., 2012)	-	34.2

B. Generalisation of very deep features

- 더 작은 데이터셋에서의 특징 추출기로서 평가

Net-D, Net-E 사용,

마지막 fc layer을 제거

마지막에서 두 번째 layer의 4096-D activation을 이미지 feature로 사용

resulting 이미지 표현 벡터-> L2 정규화되고, target 데이터셋에서 훈련된 선형 SVM분류기와 결합됨

미리 훈련된 ConvNEt의 weight는 유지됨

이미지 전체에 밀집하게 네트워크가 적용됨

B. Generalisation of very deep features

- 더 작은 데이터셋에서의 특징 추출기로서 평가

feature map 결과에서 전체 평균 pooling을 진행함 → 4096-D 이미지 descriptor를 만듦

좌우반전된 이미지의 descriptor와 같이 평균화

여러 scale Q에서 feature를 추출 → scale 전체에서 Stacking or pooling

* Stacking → 순차적 분류기가 scale 범위에서 최적으로 이미지 통계를 집계하는법을 배우도록 함

→ 두 개의 네트워크에서 계산된 특징을 결합하는 late feature의 융합

→ 그러나 증가된 차원으로 인해 비용 발생

Image Classification on VOC-2007 and VOC-2012

- PASCAL VOC-2007 and VOC-2012 benchmarks
 - 이 데이터셋은 각각 10K, 22.5K 이미지를 포함함
 - 각 이미지는 1개 또는 20 객체 카테고리에 상응하는 여러개의 label로 붙여짐
- 인지 성능은 mean average precision(mAP)로 측정됨

Table 11: **Comparison with the state of the art in image classification on VOC-2007, VOC-2012, Caltech-101, and Caltech-256.** Our models are denoted as “VGG”. Results marked with * were achieved using ConvNets pre-trained on the *extended* ILSVRC dataset (2000 classes).

Method	VOC-2007 (mean AP)	VOC-2012 (mean AP)	Caltech-101 (mean class recall)	Caltech-256 (mean class recall)
Zeiler & Fergus (Zeiler & Fergus, 2013)	-	79.0	86.5 ± 0.5	74.2 ± 0.3
Chatfield et al. (Chatfield et al., 2014)	82.4	83.2	88.4 ± 0.6	77.6 ± 0.1
He et al. (He et al., 2014)	82.4	-	93.4 ± 0.5	-
Wei et al. (Wei et al., 2014)	81.5 (85.2*)	81.7 (90.3*)	-	-
VGG Net-D (16 layers)	89.3	89.0	91.8 ± 1.0	85.0 ± 0.2
VGG Net-E (19 layers)	89.3	89.0	92.3 ± 0.5	85.1 ± 0.3
VGG Net-D & Net-E	89.7	89.3	92.7 ± 0.5	86.2 ± 0.3

Image Classification on Caltech-101 and Caltech-256

- Caltech-101 → 102 class로 라벨링된 9K 이미지들을 가짐
- Caltech-256 → 257 class로 라벨링된 31K 이미지
- 학습용과 테스트용 데이터로 여러 번 무작위 분할을 수행한 뒤, splits 전체 평균 인지 성능을 기록

Method	VOC-2007 (mean AP)	VOC-2012 (mean AP)	Caltech-101 (mean class recall)	Caltech-256 (mean class recall)
Zeiler & Fergus (Zeiler & Fergus, 2013)	-	79.0	86.5 ± 0.5	74.2 ± 0.3
Chatfield et al. (Chatfield et al., 2014)	82.4	83.2	88.4 ± 0.6	77.6 ± 0.1
He et al. (He et al., 2014)	82.4	-	93.4 ± 0.5	-
Wei et al. (Wei et al., 2014)	81.5 (85.2*)	81.7 (90.3*)	-	-
VGG Net-D (16 layers)	89.3	89.0	91.8 ± 1.0	85.0 ± 0.2
VGG Net-E (19 layers)	89.3	89.0	92.3 ± 0.5	85.1 ± 0.3
VGG Net-D & Net-E	89.7	89.3	92.7 ± 0.5	86.2 ± 0.3

Action Classification on VOC-2012

- Net-D와 Net-E의 특징을 스택한 표현을 VOC-2012 액션 분류 과제에 적용
사람의 **bounding box**가 주어진 상태에서 단일 이미지로부터 액션 클래스를 예측
데이터셋은 11개 클래스에 대해 4,600장의 학습 이미지를 포함하며, 성능은 mAP로 측정됨
- 학습 설정
 - 전체 이미지에서 ConvNet 특징을 계산하고 **bounding box**는 무시하는 방식
 - 전체 이미지와 **bounding box** 영역 모두에서 특징을 계산한 뒤 이를 스택하여 최종 표현으로 사용하는 방식

Action Classification on VOC-2012

- 결과

Table 12: **Comparison with the state of the art in single-image action classification on VOC-2012.** Our models are denoted as “VGG”. Results marked with * were achieved using ConvNets pre-trained on the *extended* ILSVRC dataset (1512 classes).

Method	VOC-2012 (mean AP)
(Oquab et al., 2014)	70.2*
(Gkioxari et al., 2014)	73.6
(Hoai, 2014)	76.3
VGG Net-D & Net-E, image-only	79.2
VGG Net-D & Net-E, image and bounding box	84.0

Other recognition Task

- Girshick et al. (2014)은 Krizhevsky et al. (2012)의 ConvNet을 VGG의 16-layer 모델로 교체함으로써 객체 검출 성능에서 최고 수준의 결과를 달성
- 의미 분할(Long et al., 2014), 이미지 캡션 생성(Kiros et al., 2014; Karpathy & Fei-Fei, 2014), 텍스처 및 소재 인식(Cimpoi et al., 2014; Bell et al., 2014)에서도 성능 향상됨

END